



Estruturas de Repetição (For)

:::: Introdução a Algoritmos

Concatenação de Strings

Em Python, podemos juntar textos usando o operador +:

```
nome = "Ana"  
sobrenome = "Silva"  
  
nome_completo = nome + sobrenome  
print(nome_completo)
```

Resultado?

AnaSilva

Concatenação de Strings

Em Python, podemos juntar textos usando o operador +:

```
nome = "Ana"
sobrenome = "Silva"

nome_completo = nome + " " + sobrenome
print(nome_completo)
```

Resultado?

Ana Silva

Construindo *strings* Dinamicamente

Também podemos montar um texto aos poucos:

```
texto = ""  
texto += "A"  
texto += "B"  
texto += "C"  
  
print(texto)
```

Resultado?

ABC



Agora, é sua vez!

Em sistemas de segurança, como caixas eletrônicos e aplicativos bancários, o acesso do usuário costuma ser protegido por um **PIN (Personal Identification Number)**.

Por questões de segurança e interface, esse PIN muitas vezes é digitado um dígito por vez, e o sistema precisa armazenar e reconstruir o valor completo a partir dessas entradas individuais. Nosso objetivo é simular esse comportamento:

- ler 4 dígitos (um por vez)
- montar o PIN completo a partir desses dígitos
- exibir o resultado final

Pensando no Problema

Entrada:

- um dígito por vez

Processamento:

- repetir a leitura 4 vezes
- juntar os dígitos

Saída:

- PIN completo

Casos de Teste

Dígitos	PIN
---------	-----

1

2

3

4

PIN digitado: 1245

" "

" "

" "

" "

PIN digitado: " "

1

" "

2

3

PIN digitado: 1 23

Implementação



```
contador = 1
```

```
pin = ""
```

```
while contador <= 4:
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
    contador += 1
```

```
print(f"PIN digitado: {pin}")
```

Recaptulando



```
contador = 1
```

```
pin = ""
```

```
while contador <= 4:
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
    contador += 1
```

```
print(f"PIN digitado: {pin}")
```

Contador: controla as repetições

Condição: define quando parar

Concatenação: constrói o resultado

Atualização: evita loop infinito

Padrão do **while** com Contador

Se abstrairmos o código, ele segue uma estrutura que se repete em muitos problemas:

```
contador = 1

while contador <= limite:
    # ação
    contador += 1
```

Risco: Loop Infinito

Se o contador não for atualizado:

- ele nunca muda
- a condição nunca se torna falsa
- o loop nunca termina

Esse é um erro muito comum em programação!

Limitação do `while`

1. Controle espalhado
2. Depende do programador
3. Fácil de errar

Podemos automatizar isso?

Uma Nova Forma de Repetição

Para repetições com **quantidade definida**, Python oferece uma estrutura própria.

Essa estrutura simplifica o padrão que antes fazíamos manualmente com `while`.

Essa estrutura é o:

`for`

Primeiro Exemplo com **for**



```
for contador in range(1, 6):  
    print(contador)
```

Resultado?

1
2
3
4
5

Note que:

- `contador` assume valores automaticamente
- `range(1, 6)` gera: 1, 2, 3, 4, 5
- o bloco executa **5 vezes**

Primeiro Exemplo com **for**

Atenção: indentação → bloco

```
for contador in range(1, 6):  
    print(contador)
```

Lê-se: para cada valor de `contador` no intervalo de 1 até 5, execute o bloco!

while X for



```
contador = 1
```

```
while contador <= 5:  
    print(contador)  
    contador += 1
```



```
for contador in range(1, 6):  
    print(contador)
```

while X for



```
contador = 1
```

```
while contador <= 5:  
    print(contador)  
    contador += 1
```



```
for contador in range(1, 6):  
    print(contador)
```



Observe a diferença:

- não precisamos criar o **contador** manualmente
- não precisamos escrever a **condição**
- não precisamos **atualizar** o contador

Controle automático do loop → **menos chance de erro**

O Papel do `range()`

A função `range(...)` define os valores da repetição

Por exemplo, `range(1, 6)` produz: 1, 2, 3, 4, 5

Ele organiza:

- onde começa
- até onde vai
- como avança

No `for`, esse padrão fica concentrado em uma única estrutura!

Estrutura Geral do `range()`

A função `range` pode ser escrita assim:

```
range(início, fim, passo)
```

Onde:

- início → valor inicial (está incluso no intervalo)
- fim → limite da sequência (**não** está incluso no intervalo)
- passo → define de quanto em quanto avança (incremento)

Exemplos de Uso do `range()`

```
>>> range(1, 5)
```

```
1, 2, 3, 4
```

```
>>> range(6, 10)
```

```
6, 7, 8, 9
```

```
>>> range(1, 10, 2)
```

```
1, 3, 5, 7, 9
```



Agora, é sua vez!

Em sistemas de segurança, como caixas eletrônicos e aplicativos bancários, o acesso do usuário costuma ser protegido por um **PIN (Personal Identification Number)**.

Por questões de segurança e interface, esse PIN muitas vezes é digitado um dígito por vez, e o sistema precisa armazenar e reconstruir o valor completo a partir dessas entradas individuais. Nosso objetivo é simular esse comportamento:

- ler 4 dígitos (um por vez)
- montar o PIN completo a partir desses dígitos
- exibir o resultado final

Reescreva a solução usando `for` no lugar do `while`

Implementação com **for**



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):  
    digito = input(f"Digite o dígito {contador}: ")  
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

Memória



O que acontece na execução?

Terminal

```
pin = ""
```

```
for contador in range(1, 5):  
    digito = input(f"Digite o dígito {contador}: ")  
    pin += digito  
  
print(f"PIN digitado: {pin}")
```

Memória



pin

O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o digito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

Memória



pin

contador

O que acontece na execução?

Terminal

```
pin = ""

for contador in range(1, 5):
    digito = input(f"Digite o dígito {contador}: ")
    pin += digito

print(f"PIN digitado: {pin}")
```

>>> Digite o dígito 1:

Memória



O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

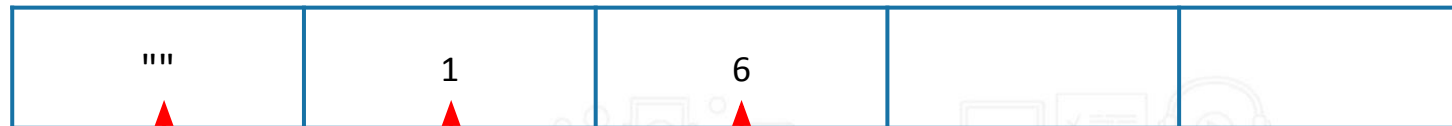
```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

Memória



pin

contador

digito

O que acontece na execução?

Terminal

```
pin = ""

for contador in range(1, 5):
    digito = input(f"Digite o dígito {contador}: ")
    pin += digito

print(f"PIN digitado: {pin}")
```

>>> Digite o dígito 1: 6

Memória



O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

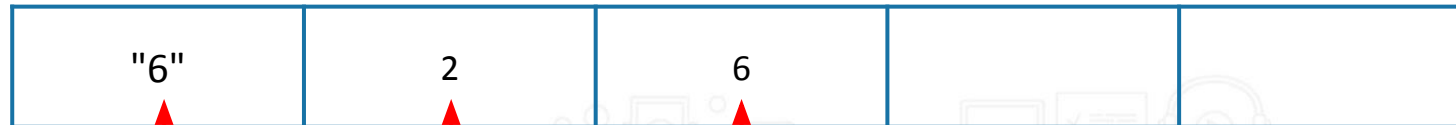
```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

Memória



pin

contador

digito

O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

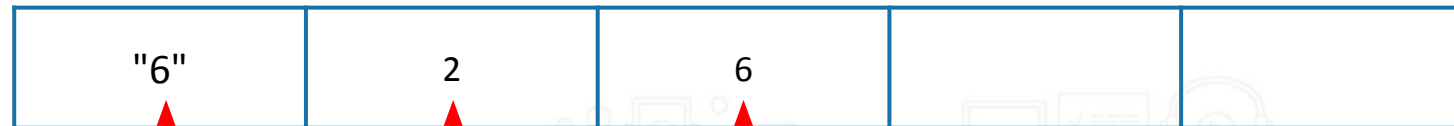
```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

```
>>> Digite o dígito 2:
```

Memória



pin

contador

digito

O que acontece na execução?

Terminal

```
pin = ""

for contador in range(1, 5):
    digito = input(f"Digite o dígito {contador}: ")
    pin += digito

print(f"PIN digitado: {pin}")
```

>>> Digite o dígito 1: 6
>>> Digite o dígito 2: 7

Memória



O que acontece na execução?

Terminal

```
pin = ""

for contador in range(1, 5):
    digito = input(f"Digite o dígito {contador}: ")
    pin += digito

print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
>>> Digite o dígito 2: 7
```

Memória



O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

```
>>> Digite o dígito 2: 7
```

Memória

"67"	3	7		
------	---	---	--	--

pin

contador

digito

O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

```
>>> Digite o dígito 2: 7
```

```
>>> Digite o dígito 3:
```

Memória

"67"	3	7		
------	---	---	--	--

pin

contador

digito

O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

```
>>> Digite o dígito 2: 7
```

```
>>> Digite o dígito 3: 8
```

Memória

"67"	3	8		
------	---	---	--	--

pin

contador

digito

O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

```
>>> Digite o dígito 2: 7
```

```
>>> Digite o dígito 3: 8
```

Memória

"678"	3	8		
-------	---	---	--	--

pin

contador

digito

O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

```
>>> Digite o dígito 2: 7
```

```
>>> Digite o dígito 3: 8
```

Memória

"678"	4	8		
-------	---	---	--	--

pin

contador

digito

O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

```
>>> Digite o dígito 2: 7
```

```
>>> Digite o dígito 3: 8
```

```
>>> Digite o dígito 4:
```

Memória

"678"	4	8		
-------	---	---	--	--

pin

contador

digito

O que acontece na execução?

Terminal



```
pin = ""
```

```
for contador in range(1, 5):
```

```
    digito = input(f"Digite o dígito {contador}: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
```

```
>>> Digite o dígito 2: 7
```

```
>>> Digite o dígito 3: 8
```

```
>>> Digite o dígito 4: 9
```

Memória

"678"	4	9		
-------	---	---	--	--

pin

contador

digito

O que acontece na execução?

Terminal

```
pin = ""

for contador in range(1, 5):
    digito = input(f"Digite o dígito {contador}: ")
    pin += digito

print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
>>> Digite o dígito 2: 7
>>> Digite o dígito 3: 8
>>> Digite o dígito 4: 9
```

Memória



O que acontece na execução?

Terminal

```
pin = ""

for contador in range(1, 5):
    digito = input(f"Digite o dígito {contador}: ")
    pin += digito

print(f"PIN digitado: {pin}")
```

```
>>> Digite o dígito 1: 6
>>> Digite o dígito 2: 7
>>> Digite o dígito 3: 8
>>> Digite o dígito 4: 9
PIN digitado: 6789
```

Memória



O que mudou em relação ao **while**?



```
contador = 1
pin = ""

while contador <= 4:
    digito = input(f"Digite o dígito {contador}: ")
    pin += digito
    contador += 1

print(f"PIN digitado: {pin}")
```



```
pin = ""

for contador in range(1, 5):
    digito = input(f"Digite o dígito {contador}: ")
    pin += digito

print(f"PIN digitado: {pin}")
```

O que mudou em relação ao **while**?



```
contador = 1  
pin = ""
```

```
while contador <= 4:  
    digito = input(f"Digite o dígito {contador}: ")  
    pin += digito  
    contador += 1
```

```
print(f"PIN digitado: {pin}")
```



```
pin = ""
```

```
for contador in range(1, 5):  
    digito = input(f"Digite o dígito {contador}: ")  
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

Consequência Prática

O uso do `for` torna o código:

- mais previsível
- mais seguro
- mais fácil de entender

Especialmente em repetições com **quantidade fixa**



Agora, é sua vez!

Em sistemas de segurança, como caixas eletrônicos e aplicativos bancários, o acesso do usuário costuma ser protegido por um **PIN (Personal Identification Number)**.

Por questões de segurança e interface, esse PIN muitas vezes é digitado um dígito por vez, e o sistema precisa armazenar e reconstruir o valor completo a partir dessas entradas individuais. Nosso objetivo é simular esse comportamento:

- ler 4 dígitos (um por vez)
- montar o PIN completo a partir desses dígitos
- exibir o resultado final

Reescreva a solução, verificando se o dígito é válido (entre 0 e 9)

Implementação com **for** + **while**

```
pin = ""

for contador in range(1, 5):
    digito = input(f"Digite o dígito {contador}: ")

    while int(digito) < 0 or int(digito) > 9:
        digito = input(f"Dígito inválido. Digite um valor entre 0 e 9: ")

    pin += digito

print(f"PIN digitado: {pin}")
```

Implementação com **for** + **while**



```
pin = ""
```

```
for contador in range(1, 5):  
    digito = input(f"Digite o dígito {contador}: ")
```

```
    while int(digito) < 0 or int(digito) > 9:  
        digito = input(f"Dígito inválido. Digite um valor entre 0 e 9: ")
```

```
    pin += digito
```

```
print(f"PIN digitado: {pin}")
```

Por que o **while** faz mais sentido aqui do que o **for**?



Agora, é sua vez!

Escreva um programa que:

- leia o nome de uma pessoa
- mostre cada letra do nome em uma linha separada

Exemplo:

Entrada: Ana

Saída:

A

n

a

O que muda nesse problema?

1. Não temos um limite fixo
2. Não faz sentido usar apenas contagem
3. Agora precisamos percorrer uma sequência

Uma Nova Forma de Usar o **for**

```
nome = input("Digite seu nome: ")

for letra in nome:
    print(letra)
```

Note que:

- `letra` assume cada caractere da string
- o loop percorre automaticamente o texto

Uma Nova Forma de Usar o **for**



```
nome = input("Digite seu nome: ")  
  
for letra in nome:  
    print(letra)
```

Lê-se: para cada `letra` em `nome`,
execute o bloco!

O que mudou?

Agora o `for` não está mais:

- contando repetições
- usando range

Ele está percorrendo uma **sequência**!

Novo Modelo de Repetição

O `for` pode ser usado de duas formas:

- com `range` → repetição por quantidade
- com sequência → repetição por elementos

Leitura conceitual:

- o loop percorre diretamente os dados
- cada iteração trabalha com um elemento
- não depende de contador

Comparando os Modelos

Agora temos três formas de controlar repetição:

Tipo	Controle	Quando usar?
<code>while</code>	depende de uma condição lógica	quando não sabemos quantas vezes repetir
<code>for + range</code>	depende de quantidade	quando sabemos a quantidade de repetições
<code>for (sequência)</code>	depende dos dados	quando queremos percorrer dados

“Se o *while* resolve tudo, por que usar *for*?”

1. evitar erros, como esquecer o incremento
2. expressar melhor a intenção do código
3. facilitar a leitura
4. reduzir a quantidade de código

Isto é:

- com *while*, escrevemos mais o mecanismo
- com *for*, escrevemos mais a intenção

Ideia Central da Aula

Cada estrutura tem seu papel:

- `while` → controle manual
- `for` → controle estruturado

Escolher a estrutura certa torna o código melhor!

Desafio Final: Cadastro de Usuários

Um sistema de cadastro precisa validar nomes antes de armazená-los. Para cada usuário, o sistema deve solicitar o nome e garantir que ele seja válido. Um nome é considerado válido quando possui pelo menos 3 caracteres. Caso o nome seja inválido, o sistema deve informar o erro e solicitar novamente até que um nome válido seja digitado.

Após validar o nome, o sistema deve analisar cada letra e contar quantas vogais ele possui. Ao final, deve exibir o nome digitado e a quantidade de vogais encontradas. O sistema deve repetir esse processo para 5 usuários.

Exercícios de Fixação

Exercício 1. Sabendo que os alunos de uma turma com NA fizeram NP provas, faça um programa para ler o valor de NA e NP. Em seguida, ler cada uma das NP provas feitas por cada um dos NA alunos e mostrar na tela (i) a média de cada aluno, (ii) a média da turma e o (iii) percentual dos alunos cuja média foi maior ou igual a 80%.

Exercício 2. Sabendo que uma loja possui NP produtos e que, para cada produto, foram registradas NV vendas ao longo do mês, faça um programa para ler a quantidade vendida em cada uma das NV vendas de cada um dos NP produtos e mostrar na tela (i) o total vendido de cada produto, (ii) a média de vendas por produto e (iii) o percentual de produtos cujo total vendido foi maior ou igual a 100 unidades.



PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.

Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.